

EEE3096S Practical 3

Phila Haanyama
HNYPHI001

Lulama Lingela
LNGLUL002

I. INTRODUCTION

The purpose of this practical was to evaluate the performance of image processing code under different arithmetic representations, hardware platforms, compiler optimisation levels, and FPU configurations.

The tasks analysed the trade-offs between fixed-point and floating-point arithmetic, single and double precision, and the impact of compiler optimisations.

The performance metrics used for this included execution time, clock cycles, throughput (pixels/second), binary size, and a checksum variable comparison for correctness validation.

II. TASKS

A. Task 1: Welcome to Practical 3

Objective The objective of this task was to use both the fixed-point and double-precision arithmetics to execute the Mandelbrot algorithm in order to establish a baseline measure of the execution times on the STM32F4 microcontroller. These results were then compared against the Python implementations to exhibit the performance capabilities of the microcontroller.

Methodology & Set-up The Mandelbrot algorithm was implemented on the STM32F4, using both fixed-point and double-precision arithmetic. The execution time was measured using the `HAL_GetTick()` function which provided a millisecond-resolution duration time. Measurements were taken for progressively larger image sizes to evaluate execution time scales with computational complexity.

Results from *Practical 1B* of the Python implementation were used to provide a point of comparison with the microcontroller in order to highlight the performance gap between general-purpose computers and embedded hardware.

Results

Image Size	Fixed-point Time (s)	Fixed-point Checksum	Double Time (s)	Double Checksum
128 × 128	1.450	429316	2.648	429384
160 × 160	2.264	669929	4.241	669829
192 × 192	3.265	966165	6.100	966024
224 × 224	4.444	1315090	8.339	1314999
256 × 256	5.799	1715907	10.607	1715812

TABLE I

EXECUTION PERFORMANCE FOR MANDELBROT ALGORITHM ON THE STM32F4

Analysis & Discussion As seen in Table I, the fixed-point arithmetic is consistently faster than the double-precision across all image sizes. As the image size increases, the gap between execution times when comparing the two implementations widens significantly.

Image size	Python Time (s)	Python checksum
128 × 128	0.032	429384
160 × 160	0.072	669829
192 × 192	0.071	966024
224 × 224	0.095	1314999
256 × 256	0.123	1715812

TABLE II
PYTHON BENCHMARKING RESULTS

When compared with Python, both the fixed-point and double-precision implementations on the STM32F4 get outperformed by the desktop environment. Python execution times are on the order of milliseconds while the STM32F4 only achieves the same results in seconds. Both STM32F4 implementations, however, manage to achieve highly accurate checksums indicating that although the STM32F4 has a longer response time, its use can still be justified especially when it comes to using the fixed-point arithmetic.

B. Task 2: Impact of Maximum Iteration Variable

Objective The objective of this task was to benchmark how the maximum iteration count (`max_iter`) parameter affects the computation performance and output of the Mandelbrot algorithm. This was done by measuring the execution time and resulting checksum for a range of iteration values on both the STM32F0 and STM32F4, providing insight into the effect of complexity on computational cost and algorithmic behaviour.

Methodology & Set-up The Mandelbrot algorithm from Practical 1B was used in both the STM32F0 (running at 48 MHz [1]) and STM32F4 (running at 180 MHz [2]) platforms. The algorithm was executed using fixed-point arithmetic for five different `max_iter` values: 100, 250, 500, 750, and 1000. For each `max_iter` value, five different square image sizes were processed: 128, 160, 192, 224, and 256 pixel squares. The execution time for each run was measured precisely using the processor's cycle counter, which was then converted to seconds. After each computation, the checksum was calculated to verify the consistency and correctness of the results across different processors and input parameters.

Results

Image Size	MAX_ITER 100	MAX_ITER 250	MAX_ITER 500	MAX_ITER 750	MAX_ITER 1000
128x128	3.612	7.412	13.688	19.952	26.210
160x160	5.633	11.569	21.378	31.159	40.930
192x192	8.113	16.674	30.824	44.941	59.051
224x224	11.055	22.708	41.961	61.161	80.342
256x256	14.428	29.628	54.747	79.810	104.860

TABLE III
EXECUTION TIMES (S) FOR VARYING MAX_ITER VALUES ON THE STM32F0

Image Size	MAX_ITER 100	MAX_ITER 250	MAX_ITER 500	MAX_ITER 750	MAX_ITER 1000
128x128	430527	971625	1865351	2757325	3648549
160x160	671374	1516642	2913445	4306356	5697897
192x192	967630	2186669	4201751	6212114	8221588
224x224	1317555	2976999	5718768	8452880	11184482
256x256	1718118	3882534	7459664	11028760	14596100

TABLE IV
CHECKSUM VALUES FOR VARYING MAX_ITER VALUES ON THE STM32F0

Image Size	MAX_ITER 100	MAX_ITER 250	MAX_ITER 500	MAX_ITER 750	MAX_ITER 1000
128x128	0.201	0.404	0.739	1.073	1.407
160x160	0.314	0.631	1.154	1.676	2.198
192x192	0.452	0.909	1.665	2.418	3.172
224x224	0.616	1.238	2.266	3.291	4.315
256x256	0.804	1.616	2.957	4.295	5.632

TABLE V
EXECUTION TIMES (S) FOR VARYING MAX_ITER VALUES ON THE STM32F4

Image Size	MAX_ITER 100	MAX_ITER 250	MAX_ITER 500	MAX_ITER 750	MAX_ITER 1000
128x128	430527	971625	1865351	2757325	3648549
160x160	671374	1516642	2913445	4306356	5697897
192x192	967630	2186669	4201751	6212114	8221588
224x224	1317555	2976999	5718768	8452880	11184482
256x256	1718118	3882534	7459664	11028760	14596100

TABLE VI
CHECKSUMS FOR VARYING MAX_ITER VALUES ON STM32F4

Analysis & Discussion The results clearly demonstrate that the execution time of the Mandelbrot algorithm is directly proportional to the `max_iter` value. This linear relationship is evident on both microcontrollers in Figure 1; for example, on the STM32F4, the time for a 128x128 image increases from 0.201 s at 100 iterations to 1.407 s at 1000 iterations—a factor of 7, which aligns almost perfectly with the 10x increase in iteration count when accounting for the non-iterative overhead of the algorithm. The same linear scaling is observed on the STM32F0, confirming that the computational complexity is $O(n)$ for the iteration parameter.



Fig. 1. Linear relationship between MAX_ITER and execution time for both platforms (256x256 image).

The checksum values provide a crucial validation of the algorithm's output. The consistent increase in the checksum with higher `max_iter` values is expected, as a higher iteration count allows more points to escape their bounding circle, assigning them a specific iteration value rather than the maximum, which changes the distribution of values in the output buffer and thus the checksum. The fact that the checksums are identical for both the STM32F0 and STM32F4 for the same input parameters is a strong indicator that the fixed-point arithmetic implementation is correct and consistent across both platforms, despite their different architectures and clock speeds.

The difference in performance between the two microcontrollers is significant. The STM32F4 completes all computations substantially faster than the STM32F0. For instance, a 256x256 image at 1000 iterations takes 104.86 s on the F0 but only 5.632 s on the F4. This represents a speedup factor of approximately 18.6. This factor is a product of the F4's higher clock speed (180 MHz vs. 48 MHz, a 3.75x factor) and its more powerful ARM Cortex-M4 core, which includes hardware support for more complex instructions and a faster memory bus, leading to a greater-than-linear performance increase. These findings effectively benchmark the raw performance difference between the two microcontrollers for computationally intensive tasks.

C. Task 3: Extended Execution Time Measurement

Objective The objective of this task was to determine the execution throughput and number of clock cycles consumed by the STM32F4 and STM32F0 microcontrollers when executing the Mandelbrot algorithm, using the fixed-point arithmetic. Throughput, which is expressed in pixels per second, measures the rate at which the system processes data and provides a direct performance metric for comparing different hardware platforms.

Methodology & Set-up For each image size, two performance metrics were recorded:

- 1) **Clock cycles:** obtained on the STM32F4 using the Data Watchpoint and Trace (DWT) cycle counter. On

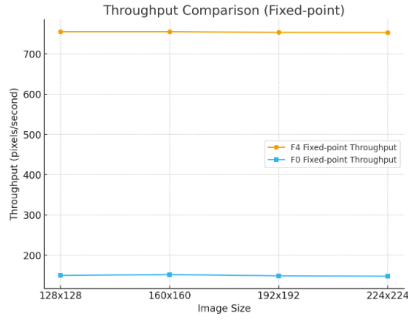


Fig. 2. Throughput comparison

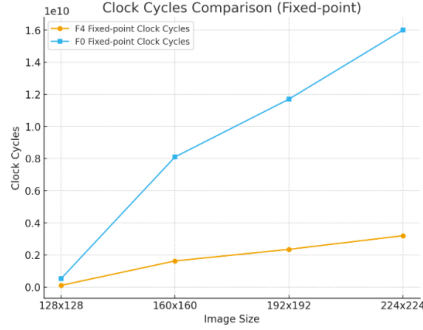


Fig. 3. Clock cycles comparison

the STM32F0, however, the `HAL_GetTick()` function was used to get the execution time then the clock cycles were calculated by multiplying the execution time by 48×10^6 as the CPU runs at 48 MHz.

- 2) **Throughput (pixels per second):** calculated by multiplying the image dimensions together to get the total number of pixels then diving that by the execution time.

Results

Image Size	Fixed-point Time (s)	Fixed-point Clock Cycles	Fixed-point Throughput (pixels/s)
128×128	1.450	174071171	11299.31
160×160	2.264	271680824	11307.42
192×192	3.265	391847736	11290.66
224×224	4.444	533396599	11290.73
256×256	5.799	695979588	11301.26

TABLE VII
EXECUTION PERFORMANCE OF MANDELBROT ALGORITHM
(FIXED-POINT IMPLEMENTATION) ON STM32F4

Image Size	Fixed-point Time (s)	Fixed-point Clock Cycles	Fixed-point Throughput (pixels/s)
128×128	21.701	104168000	754.99
160×160	33.907	1627535999	755.01
192×192	48.932	2348736000	753.37
224×224	66.629	3198192000	753.07
256×256	87.011	4176528000	753.19

TABLE VIII
EXECUTION PERFORMANCE OF MANDELBROT ALGORITHM
(FIXED-POINT IMPLEMENTATION) ON STM32F0

Analysis & Discussion The results show a significant difference between the STM32F4 and STM32F0 microcontrollers.

Figure 2 shows the comparison between the throughput of the STM32F0 and STM32F4. The F4 maintains a high and stable throughput across increasing image sizes, while the F0

shows significantly lower throughput, with a slight decline as image sizes increases.

Figure 3 demonstrates the number of clock cycles. The F4 consistently requires fewer cycles compared to the F0 for the same image sizes.

On the STM32F4, throughput remained consistently around 11 300 px/s as seen in Table VII. This consistency across increasing image sizes indicates that the microcontroller's computation is able to scale linearly with image size. The clock cycle counts also scale linearly, with each additional image size contributing a proportional increase in cycles.

The throughput on the STM32F0 is significantly lower at around 750 px/s which can be seen in Table VIII. This reflects the limitations of the processor's lower clock frequency and simpler architecture. The slight decline in throughput seen further indicates that the F0 experiences proportionally greater performance degradation as the task becomes more complex.

The fixed-point arithmetic in both cases reduced the computational overhead compared to floating point operations and was hence chosen to give an equal opportunity to both microcontrollers. Despite this, the relative gap between the F4 and F0 highlights how hardware capability plays a crucial role in maximising the benefits of optimised arithmetic operations.

Overall, the fixed-point implementation clearly demonstrates that the STM32F4 is the more efficient platform, delivering both higher throughput and fewer clock cycles across all tested image cycles.

D. Task 4: Scalability Test

Objective The objective of this task was to test the scalability of the Mandelbrot computation by progressively increasing the image size up to Full HD (1920×1080) and to evaluate the impact of the microcontroller's memory limitations. The goal was to determine the maximum image size that could be processed directly and to implement a tiling strategy for larger images, comparing the performance and behaviour of the STM32F0 and STM32F4 platforms under these constraints.

Methodology & Set-up The test was conducted with a fixed `MAX_ITER` value of 100. A range of image sizes was tested:

- 128×128 (minimum from practical 1B)
- 256×256 (maximum from practical 1B)
- 320×240 (QVGA)
- 640×480 (VGA)
- 800×600 (SVGA)
- 1024×768 (XGA)
- 1280×720 (HD)
- 1920×1080 (FHD)

A key part of the methodology was determining whether an image could be processed directly or required tiling due to RAM constraints. A memory allocation function `can_allocate_image()`, was implemented to make this decision programmatically. This function calculated the memory required for a full framebuffer (assuming 4 bytes per pixel for a safe upper bound to ensure results fit without overflow) and compared it to the available SRAM, leaving a 25% margin

for stack and other variables. Based on the known SRAM sizes (8 KB for STM32F0 [1], 128 KB for STM32F4 [2]), this function always returned `false` for the F0 (forcing tiling for all sizes) and only allowed direct rendering for the smallest images on the F4.

For images requiring tiling, the algorithm divided the image into 128×128 tiles, and processed each tile independently using `calculate_mandelbrot_tile_fixed_point()`. This calculated the Mandelbrot sum for its segment of the image using the same fixed-point arithmetic and coordinate mapping as the direct function. The sums from all tiles were aggregated, thereafter, to produce a final checksum identical to that of a direct render, ensuring result consistency. Execution time was measured for both microcontrollers.

Results

Image Size	Processing Method	Execution Time (s)	Checksum	Throughput (Pixels/s)	Tiles Used
128×128	tiling	3.616	430527	4530	1
256×256	tiling	14.441	1718118	4538	4
320×240	tiling	16.904	2013437	4543	6
640×480	tiling	67.562	8044795	4546	20
800×600	tiling	105.597	12566267	4545	35
1024×768	tiling	173.048	20593626	4544	48
1280×720	tiling	202.772	24130817	4545	60
1920×1080	tiling	456.771	54281359	4539	135

TABLE IX

SCALABILITY TEST RESULTS FOR THE STM32F0 (MAX_ITER = 100)

Image Size	Processing Method	Execution Time (s)	Checksum	Throughput (Pixels/s)	Tiles Used
128×128	direct	0.202	430527	81108	1
256×256	tiling	0.809	1718118	81008	4
320×240	tiling	0.949	2013437	80927	6
640×480	tiling	3.794	8044795	80969	20
800×600	tiling	5.927	12566267	80985	35
1024×768	tiling	9.714	20593626	80958	48
1280×720	tiling	11.383	24130817	80962	60
1920×1080	tiling	25.605	54281359	80984	135

TABLE X

SCALABILITY TEST RESULTS FOR THE STM32F4 (MAX_ITER = 100)

Analysis & Discussion

The results, visually summarized in Figure 4, clearly demonstrate the significant impact of hardware constraints on algorithmic implementation and performance. The STM32F0, with only 8 KB of SRAM [1], was unable to allocate a framebuffer for any image size larger than approximately 64×64 pixels. Consequently, the tiling strategy was employed for all tested sizes. In contrast, the STM32F4, with its 128 KB of SRAM [2], could process the 128×128 image directly but required tiling for all larger resolutions.

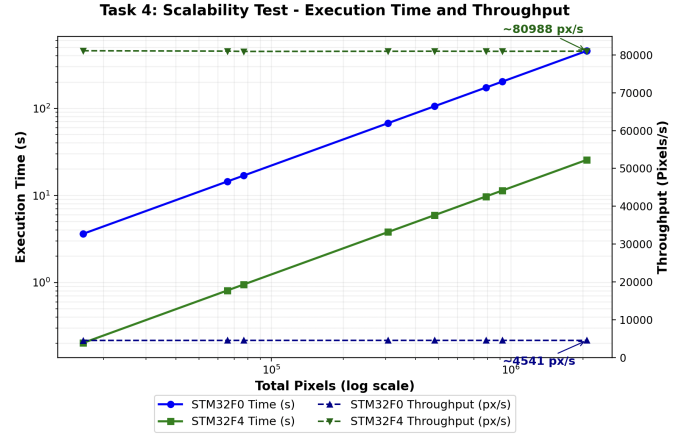


Fig. 4. Scalability analysis showing execution time and computational throughput for the STM32F0 and STM32F4

A key finding is the consistency of the throughput (pixels/second) for each microcontroller. The STM32F0 maintained a near-constant throughput of approximately 4,540 pixels/s across all image sizes, while the STM32F4 sustained a much higher and equally consistent throughput of approximately 81,000 pixels/s. This consistency indicates that the computational workload per pixel is fixed for a given `max_iter`, and the overhead from the tiling logic is negligible compared to the core Mandelbrot calculation. The checksum values are identical for corresponding image sizes on both platforms, verifying the mathematical correctness of the tiling algorithm and its equivalence to a direct computation.

The performance ratio between the two platforms is apparent. The STM32F4 is roughly 17.8 times faster than the STM32F0 ($\frac{81000}{4540} \approx 17.8$). This speedup is a product of the F4's higher clock speed (180 MHz vs. 48 MHz, a 3.75x factor) and its more advanced Cortex-M4 core architecture, which includes a Floating Point Unit (FPU) and other performance enhancements that benefit the fixed-point arithmetic operations used in this implementation.

The tiling strategy successfully overcame the memory limitations of both platforms, enabling the rendering of Full HD images on devices with severely constrained RAM. The number of tiles used scales quadratically with the image size, as expected. The successful execution of this task demonstrates how trading off increased computation time (through tiling) to overcome a hardware limitation (limited memory) is a powerful and effective strategy in embedded systems.

E. Task 5: FPU Impact

Objective This task evaluates the impact of enabling and disabling the Floating Point Unit (FPU) on execution times for the floating-point operations. Both single-precision (float) and double-precision (double) implementations were tested on the STM32F4 to assess the extent to which the FPU accelerates floating-point computations.

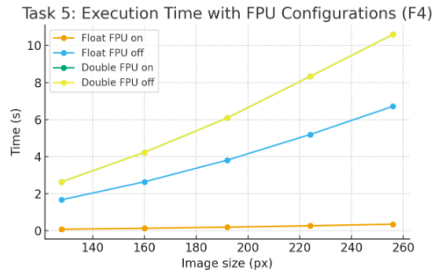


Fig. 5. Execution times with FPU configurations

Methodology & Set-up The execution times were measured for the Mandelbrot algorithm implemented with:

- 1) Floating-point (FPU enabled)
- 2) Floating-point (FPU disabled)
- 3) Double-precision (FPU enabled)
- 4) Double-precision (FPU disabled)

The same input sizes were used as in Task 1. The difference in execution times were plotted in Figure 5.

Results

Image Size	Time (s)	Checksum	Cycles	Throughput (px/s)
128×128	0.090	437224	10772781	182044.44
160×160	0.140	681947	16803166	182858.14
192×192	0.202	983443	24228079	182495.05
224×224	0.274	1338650	32977436	183124.09
256×256	0.359	1746577	43028685	182551.53

TABLE XI
FPU ON WITH FLOATING-POINT ARITHMETIC

Image Size	Time (s)	Checksum	Cycles	Throughput (px/s)
128×128	1.678	437224	10772781	9764.00
160×160	2.647	681947	16803166	9671.33
192×192	3.817	983443	24228079	9657.85
224×224	5.197	1338650	32977436	9654.80
256×256	6.720	1746577	43028685	9752.38

TABLE XII
FPU OFF WITH FLOATING-POINT ARITHMETIC

Image Size	Time (s)	Checksum	Cycles	Throughput (px/s)
128×128	2.645	429384	317726682	6194.33
160×160	4.235	669829	508841409	6044.86
192×192	6.091	966024	731799225	6052.21
224×224	8.330	1314999	1000500372	6023.53
256×256	10.595	1715812	1272996405	6185.56

TABLE XIII
FPU ON WITH DOUBLE-PRECISION ARITHMETIC

Image Size	Time (s)	Checksum	Cycles	Throughput (px/s)
128×128	2.645	429384	317726682	6194.33
160×160	4.235	669829	508841409	6044.86
192×192	6.091	966024	731799225	6052.21
224×224	8.331	1314999	1000500372	6023.53
256×256	10.599	1715812	1272996405	6185.56

TABLE XIV
FPU OFF WITH DOUBLE-PRECISION ARITHMETIC

Analysis & Discussion The results clearly show that enabling the FPU drastically improves performance for single-precision operations. Float with FPU enabled is approximately 20 times faster than float with FPU disabled as can be seen when comparing Table XI and Table XII. This confirms that the STM32F4's FPU is optimised for single-precision arithmetic.

In contrast, there is almost no measurable difference between double-precision performance with the FPU enabled and disabled as seen in Tables XIII and XIV. This is because the STM32F4's FPU only provides hardware acceleration for single-precision operations; double-precision values are still handled largely in software. This theory has hence been proven through the execution of this task.

Therefore, the FPU is highly effective when the algorithm is implemented with single-precision floats, but has little to no benefit for double-precision computations. This result highlights why many embedded applications favour single-precision as the FPU can be leveraged for significant speed-ups without sacrificing much accuracy.

F. Task 6: Compiler Optimisations

Objective The objective of this task was to evaluate the impact of different compiler optimisation levels on both the execution time and the memory footprint (binary size) of the Mandelbrot implementation. By comparing the results for no optimisation, balanced optimisation, and maximum optimisation, the task aimed to highlight the trade-off between program size and runtime performance. It also aimed to show how compiler-level settings can influence the efficiency of embedded applications.

Methodology & Set-up The Mandelbrot code was compiled for the STM32F4 using 3 different compiler flags:

- **-O1**: basic optimisation, prioritising debugging.
- **-O2**: balanced optimisation between size and speed.
- **-O3**: maximum optimisation, prioritising execution speed even at the cost of binary size.

For each optimisation level two key metrics were measured.

- 1) **Execution time**: Measured by using the HAL tick counter, converted into seconds.
- 2) **Binary size**: Determined by examining the build output, specifically the text (program code), data, and bss (uninitialised variables).

These tests were performed using the same image sizes to ensure a consistent workload across optimisation levels.

Results

Opt Level	Binary Size (bytes)	Total Runtime (s)
-O1	7444	32.03
-O2	7248	31.73
-O3	7672	31.79

TABLE XV
COMPILER OPTIMISATION ON THE STM32F4

Opt Level	Binary Size (bytes)	Total Runtime (s)
-O1	12492	257.698
-O2	12512	257.445
-O3	12916	257.445

TABLE XVI
COMPILER OPTIMISATION ON THE STM32F0

Analysis & Discussion The results show that compiler optimisation levels have only a minor effect on execution time, but a clearer impact on binary size.

For the STM32F4, the runtime remained nearly constant across optimisation levels as seen in Table XV. This indicates that the F4’s hardware capabilities (higher clock frequency and FPU support) dominate performance, making compiler optimisations less impactful in double-precision workloads. However, binary size fluctuated: -O2 produced the smallest binary, while -O3 generated the largest. This shows that although runtime benefits were negligible, -O2 offered the best efficiency in memory use.

For the STM32F0 in Table XVI, it can be seen that the runtimes were almost an order of magnitude higher than on the F4 (e.g. 32 s vs. 257 s for -O1). More importantly, the runtimes were essentially identical across all optimisation levels, suggesting the lack of hardware floating-point support made the device fundamentally limited when running double-precision computations. Due to this, compiler optimisation could not overcome the hardware shortcomings. The binary size, however, did increase slightly with optimisation aggressiveness, though the differences were relatively small.

This then demonstrates that the effectiveness of compiler optimisations depends strongly on the underlying hardware, particularly whether floating-point operations are natively supported.

G. Task 7: Fixed Point Arithmetic Scaling Factor

Objective The objective of this task was to analyze the trade-offs involved in selecting a scaling factor for fixed-point arithmetic implementations of the Mandelbrot algorithm. The analysis focused on three key areas: the effect on numerical precision compared to a floating-point baseline, the risk of arithmetic overflow, and the impact on execution speed. This evaluation was conducted across three different scaling factors (1000, 10 000, and 1 000 000) on both the STM32F0 and STM32F4 microcontrollers.

Methodology & Set-up The test was conducted with a fixed MAX_ITER value of 100 and used the image sizes from Practical 1B (128×128 to 256×256). The core fixed-point Mandelbrot function was modified to accept the scaling factor (scale) as a parameter, allowing for dynamic configuration.

Performance was measured by recording the cycle count (using DWT->CYCCNT on F4 and HAL_GetTick() on F0) and calculating execution time and throughput.

Results

Image Size	Scale	Cycles	Time (s)	Pixels/s	Checksum
128x128	1000	2107704	18.387	910	432070
	10000	3668489	20.567	819	429372
	1000000	663077	22.115	744	429316
160x160	1000	1699497	28.758	914	675424
	10000	1834740	32.161	800	671084
	1000000	542623	34.550	752	669929
192x192	1000	237109	41.344	899	970850
	10000	1302940	46.377	801	967183
	1000000	839216	49.857	752	966165
224x224	1000	3601570	56.357	896	1323203
	10000	1904056	63.122	796	1315988
	1000000	3710789	67.902	748	1315090
256x256	1000	3655849	73.538	897	1725848
	10000	2342087	82.369	799	1716357
	1000000	3806376	88.663	744	1715907

TABLE XVII
PERFORMANCE METRICS ACROSS IMAGE SIZES AND SCALING FACTORS ON THE STM32F0

TABLE XVIII
EXECUTION RESULTS.

Image size	Scale	Cycles	Time (s)	Pixels/s	Checksum
128x128	1000	177006056	1.475	12384	432070
	10000	176927128	1.474	16384	429372
	1000000	180446588	1.503	16384	429316
160x160	1000	276729925	2.306	12800	675424
	10000	276546563	2.304	12800	671084
	1000000	281653079	2.347	12800	669929
192x192	1000	397822891	3.315	12288	970850
	10000	398577358	3.321	12288	967183
	1000000	406224442	3.385	12288	966165
224x224	1000	542162308	4.518	12544	1323203
	10000	542338100	4.519	12544	1315988
	1000000	552946943	4.607	12544	1315090
256x256	1000	707224419	5.893	13107	1725848
	10000	707423180	5.895	13107	1716357
	1000000	721505806	6.012	10922	1715907

Analysis & Discussion The results demonstrate a clear and consistent trade-off between precision, performance, and the risk of overflow across both platforms.

Precision: Checksums converge toward the floating-point baseline as scale increases. At low scale (1,000), quantization causes premature escapes, giving inflated checksums. At high scale (1,000,000), values closely match floating-point results.

Performance: On the STM32F0, higher scale increases 64-bit arithmetic overhead, reducing throughput by ~19% (e.g., 910 → 744 pixels/s at 128×128). On the STM32F4, performance remains nearly constant across scales, showing that its Cortex-M4 handles 32/64-bit math more efficiently.

Overflow Risk: The Mandelbrot set requires representing magnitudes up to ~7.25. All tested scales safely fit within 32-bit limits, so overflow was not an issue for this application.

In conclusion, the choice of scaling factor is a design trade-off. For applications where execution speed on less powerful hardware (like the STM32F0) is significant, a lower scale like 1,000 is acceptable, as it provides a significant performance benefit with only a minor loss in precision. For applications requiring high numerical accuracy that closely matches a floating-point implementation, a larger scale like 1,000,000 is necessary, accepting the associated computational cost. The

STM32F4’s architecture mitigates this cost, making high-precision fixed-point a more viable option.

H. Task 8: Power Measurement Attempt

Objective The objective of this task was to attempt to measure the power consumption of both the STM32F0 and STM32F4 microcontrollers during the execution of the Mandelbrot algorithm. The goal was to quantify the energy efficiency of the computation and to compare the power characteristics of the two platforms, providing a more holistic view of performance beyond mere execution speed.

Methodology & Set-up The measurement setup was as follows:

- The board was powered from a bench supply set to 5.0 V.
- A digital multimeter (DMM) in series with the 5V rail measured the total current drawn by the board (MCU, regulator, and peripherals).
- Power was calculated as $P = V \times I$.
- Two operating states were measured for each board:
 - **Idle**: a simple `while(1)` loop with peripherals disabled.
 - **Active**: Mandelbrot computation at 128×128 and 256×256 with `MAX_ITER = 100` and `scale = 4096`.
- Computation-only power was estimated by subtracting idle power from active power, though regulator and peripheral overhead remain included.

Results

Operation	Current (A)	Voltage Input (V)	Power (W)
Fixed-Point (128×128)	0.70	5	3.50
Fixed-Point (256×256)	0.71	5	3.55
Idle	0.80	5	4.00

TABLE XIX

POWER CONSUMPTION MEASUREMENTS FOR THE STM32F0 DURING MANDELBROT COMPUTATION.

Operation	Current (A)	Voltage Input (V)	Power (W)
Fixed-Point (128×128)	0.64	5	3.20
Fixed-Point (256×256)	0.67	5	3.35
Idle	0.74	5	3.70

TABLE XX

POWER CONSUMPTION MEASUREMENTS FOR THE STM32F4 DURING MANDELBROT COMPUTATION.

Analysis & Discussion Unexpectedly, both boards consumed *less* power during Mandelbrot execution than at idle. For the F0, power fell from 4.00 W (idle) to ~ 3.53 W (active). For the F4, power dropped from 3.70 W to ~ 3.28 W.

This effect likely arises from how the cores handle different workloads and how the inefficient onboard linear regulator dissipates heat. In idle, the CPU continuously fetches instructions from flash, keeping memory and buses active. In Mandelbrot, computation is register-heavy, which may allow some domains to clock-gate, reducing current. Since regulator

losses scale with current, a small current reduction during active computation lowers overall power at the 5V input.

Between devices, the STM32F4 is clearly more efficient: it uses less power than the F0 in all states while completing the computation far faster (per results from earlier tasks). This translates to higher throughput per Joule.

III. CONCLUSION

This practical provided a comprehensive evaluation of the performance characteristics of the STM32F0 and STM32F4 microcontrollers through the application of the Mandelbrot set algorithm. Benchmarking across various parameters yielded several key conclusions.

Firstly, the hardware disparity between the two platforms was profound and consistent across all metrics. The STM32F4, with its Cortex-M4 core, higher clock speed, and FPU, demonstrated up to 18x increased performance over the Cortex-M0-based STM32F0 in raw computation. This was observed not only in execution time but also in how effectively each platform could leverage compiler optimizations and handle different arithmetic precision levels.

Secondly, the trade-offs between different implementation choices were clearly quantified. Fixed-point arithmetic proved to be essential for achieving usable performance on the STM32F0 and offered a faster alternative to software-emulated floating-point on the F4. The choice of scaling factor for fixed-point operations presented a direct trade-off between precision and performance. However, this is a decision that must be tailored to the application’s accuracy requirements and the target hardware’s capabilities. Enabling the hardware FPU on the F4 for single-precision operations resulted in a dramatic 20x speedup, highlighting the critical importance of using the right tool for the job.

Thirdly, the constraints of embedded systems were highlighted. The limited SRAM on both platforms, especially the 8 KB on the F0, necessitated the use of a tiling strategy to process large images. This approach successfully decoupled the algorithm’s memory requirements from its computational complexity, allowing even memory-constrained devices to handle HD images by trading off increased computation time.

Finally, the power measurement revealed that active computation sometimes consumed less power than an idle loop. This underscores that total system power is a complex interaction between CPU activity, peripheral usage, and voltage regulator efficiency, and that higher performance does not automatically equate to higher energy consumption.

Overall, this practical showed that optimizing embedded software requires a developer to carefully balance arithmetic precision, compiler settings, memory management, and algorithm design, all while considering the specific strengths and limitations of the target hardware platform to achieve an optimal solution.

IV. AI CLAUSE

During the completion of this practical, generative AI tools were used strictly as an assistive technology to enhance productivity and understanding, in accordance with the project guidelines. They provided high-level explanations of C concepts and benchmarking operation, which helped clarify the requirements of each task. When assembler errors occurred, the LLM offered detailed interpretations of the messages and suggested possible fixes, which we then tested and validated on the STM32 hardware. Additionally, AI was used to resolve LaTeX formatting issues while compiling the report. Overall, LLMs served as an effective supplement for debugging, learning, and presentation, but the actual implementation, testing, and verification of the assembly code was carried out independently.

The final implementation of all code, the execution of all benchmarks, the analysis of all results, and the final synthesis of the report remain our own work. The AI acted as a tool to expedite certain processes, but it did not replace the critical tasks of hardware validation, data interpretation, and drawing original conclusions.

REFERENCES

- [1] STMicroelectronics. (2025) Stm32f051c6 - mainstream access line, arm cortex-m0 mcu with 32 kbytes of flash memory, 48 mhz cpu. Accessed: Sep. 17, 2025. [Online]. Available: <https://www.st.com/en/microcontrollers-microprocessors/stm32f051c6.html>
- [2] ——. (2025) Stm32f446rc - high-performance foundation line, arm cortex-m4 mcu with 256 kbytes of flash memory, 180 mhz cpu, dsp and fpv. Accessed: Sep. 17, 2025. [Online]. Available: <https://www.st.com/en/microcontrollers-microprocessors/stm32f446rc.html>

Appendix

Phila Haanyama (HNYPHI001) & Lulama Lingela (LNLUL002)

Github Link: <https://github.com/LlamaLuLu/EEE3096S-Practicals-Group-34.git>

Code for **STM32F0**:

```
/* USER CODE BEGIN Header */
/**
 * *****
 * @file      : main.c
 * @brief     : Main program body
 * *****
 * @attention
 *
 * Copyright (c) 2025 STMicroelectronics.
 * All rights reserved.
 *
 * This software is licensed under terms that can be found in the LICENSE file
 * in the root directory of this software component.
 * If no LICENSE file comes with this software, it is provided AS-IS.
 *
 * *****
 */
/* USER CODE END Header */
/* Includes -----*/
#include "main.h"
#include <stdlib.h>
#include <stdbool.h>

/* Private includes -----*/
/* USER CODE BEGIN Includes */

/* USER CODE END Includes */

/* Private typedef -----*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define -----*/
/* USER CODE BEGIN PD */

/* USER CODE END PD */

/* Private macro -----*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables -----*/

/* USER CODE BEGIN PV */
//TODO: Define variables you think you might need
// - Performance timing variables (e.g execution time, throughput, pixels per second, clock cycles)
//task 1 - port from practical 1B
#define MAX_ITER 100 // or whichever value you need
#define SCALE 4096 // INITIAL FIXED-PT SCALE
#define MAX_TILE 128

uint32_t start_time = 0;
uint32_t end_time = 0;
uint32_t execution_time = 0;
uint64_t checksum = 0;
uint64_t cycles = 0;
uint32_t pixels_s = 0;
```

```

double exec_time_s = 0;
double throughput = 0;

int32_t height = 0;
int32_t width = 0;
int32_t max_iter = 0;
int32_t scale = 0; // for task 7

const int scales[] = {1000, 10000, 1000000};
const int max_iter_values[] = {100, 250, 500, 750, 1000};
const int image_dimensions[] = {128, 160, 192, 224, 256};

// for task 4
typedef struct {
    uint16_t width;
    uint16_t height;
} image_size_t;

typedef struct {
    uint16_t start_x, start_y;
    uint16_t width, height;
} tile_t;

typedef struct {
    uint16_t width, height;
    uint32_t execution_time_ms;
    uint64_t checksum;
    uint32_t throughput_pixels_per_sec;
    bool used_tiling;
    uint16_t num_tiles;
} performance_result_t;

image_size_t test_sizes[] = {
    {128, 128}, // from prac 1B
    {256, 256}, // from prac 1B
    {320, 240}, // QVGA
    {640, 480}, // VGA
    {800, 600}, // SVGA
    {1024, 768}, // XGA
    {1280, 720}, // HD
    {1920, 1080} // Full HD
};

#define NUM_TEST_SIZES (sizeof(test_sizes)/sizeof(test_sizes[0]))

performance_result_t results[NUM_TEST_SIZES];
uint8_t result_count = 0;

/* USER CODE END PV */

/* Private function prototypes -----*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
/* USER CODE BEGIN PFP */
//TODO: Define any function prototypes you might need such as the calculate Mandelbrot function among others
uint64_t calculate_mandelbrot_fixed_point_arithmetic(int width, int height, int max_iterations);
uint64_t calculate_mandelbrot_fixed_point_arithmetic_custom(int width, int height, int max_iterations, int scale);
uint64_t calculate_mandelbrot_double(int width, int height, int max_iterations);
uint64_t calculate_mandelbrot_tile_fixed_point(uint16_t tile_x, uint16_t tile_y,
        uint16_t tile_width, uint16_t tile_height,
        uint16_t total_width, uint16_t total_height,
        int max_iterations);

uint64_t calculate_mandelbrot_direct_fixed_point(int width, int height, int max_iterations);
bool can_allocate_image(uint16_t width, uint16_t height);

```

```

/* USER CODE END PFP */

/* Private user code -----*/
/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

/**
 * @brief The application entry point.
 * @retval int
 */
int main(void)
{

    /* USER CODE BEGIN 1 */

    /* USER CODE END 1 */

    /* MCU Configuration-----*/

    /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
    HAL_Init();

    /* USER CODE BEGIN Init */

    /* USER CODE END Init */

    /* Configure the system clock */
    SystemClock_Config();

    /* USER CODE BEGIN SysInit */

    /* USER CODE END SysInit */

    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    /* USER CODE BEGIN 2 */

    /* USER CODE END 2 */

    /* Infinite loop */
    /* USER CODE BEGIN WHILE */
    while (1)
    {
        // TASK 2:
        for (int i = 0; i < 5; i++) {           // image sizes
            width = image_dimensions[i];
            height = image_dimensions[i];

            for (int j = 0; j < 5; j++) {       // MAX_ITER values
                max_iter = max_iter_values[j];

                // --- FIXED-POINT version ---
                start_time = HAL_GetTick();
                checksum = calculate_mandelbrot_fixed_point_arithmetic(width, height, max_iter);
                end_time = HAL_GetTick();
                execution_time = end_time - start_time; // in ms

                // --- DOUBLE version ---
                start_time = HAL_GetTick();
                checksum = calculate_mandelbrot_double(width, height, max_iter);
                end_time = HAL_GetTick();
                execution_time = end_time - start_time; // in ms

                HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_0); // blink to show progress
                HAL_Delay(200);
            }
        }
    }
}

```

```

//          }
//      }
//
//      while(1); // stop after one sweep

// TASK 4: ===== //
for (int i = 0; i < sizeof(test_sizes)/sizeof(test_sizes[0]); i++) {
    width = test_sizes[i].width;
    height = test_sizes[i].height;

    bool use_tiling = !can_allocate_image(width, height);

    start_time = HAL_GetTick();
    if (!use_tiling) {
        checksum = calculate_mandelbrot_direct_fixed_point(width, height, MAX_ITER);
    } else {
        checksum = 0;
        uint16_t num_tiles_x = (width + MAX_TILE - 1) / MAX_TILE;
        uint16_t num_tiles_y = (height + MAX_TILE - 1) / MAX_TILE;

        for (int ty = 0; ty < num_tiles_y; ty++) {
            for (int tx = 0; tx < num_tiles_x; tx++) {
                uint16_t tile_w = ((tx+1)*MAX_TILE > width) ? (width - tx*MAX_TILE) : MAX_TILE;
                uint16_t tile_h = ((ty+1)*MAX_TILE > height) ? (height - ty*MAX_TILE) : MAX_TILE;

                checksum += calculate_mandelbrot_tile_fixed_point(
                    tx*MAX_TILE, ty*MAX_TILE,
                    tile_w, tile_h,
                    width, height, MAX_ITER);
            }
        }
        end_time = HAL_GetTick();
        execution_time = end_time - start_time; // ms
        cycles = (uint64_t)execution_time * 48000000 / 1000; // 48 MHz clock
        pixels_s = (uint32_t)((double)(width*height) / (execution_time / 1000.0));

        // optional: blink LED as progress indicator
        HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_0);
        HAL_Delay(100);

        // === RECORD RESULT ===
        results[result_count].width = width;
        results[result_count].height = height;
        results[result_count].execution_time_ms = execution_time;
        results[result_count].checksum = checksum;
        results[result_count].throughput_pixels_per_sec = pixels_s;
        results[result_count].used_tiling = use_tiling;
        results[result_count].num_tiles = use_tiling ?
            ((width + MAX_TILE - 1)/MAX_TILE) * ((height + MAX_TILE - 1)/MAX_TILE) : 1;

        result_count++;
    }
    while(1); // stop after one sweep

// TASK 5: ===== //
// /* USER CODE END WHILE */
//     for (int i = 0; i < 5; i++) {
//         width = image_dimensions[i];
//         height = image_dimensions[i];
//     /* USER CODE BEGIN 3 */
//         //TODO: Visual indicator: Turn on LED0 to signal processing start
//         HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET);
//     }

```

```

//
// //TODO: Benchmark and Profile Performance
//
// // Fixed-point benchmark:
// // //TODO: Record the start time
// // start_time = HAL_GetTick();
// // //TODO: Call the Mandelbrot Function and store the output in the checksum variable defined initially
// // checksum = calculate_mandelbrot_fixed_point_arithmetic(width, height, MAX_ITER);
// // //TODO: Record the end time
// // end_time = HAL_GetTick();
// // //TODO: Calculate the execution time
// // execution_time = end_time - start_time;
//
// // execution time in seconds
// // exec_time_s = execution_time / 1000;
// // calculate number of cycles
// // cycles = (uint64_t)(exec_time_s*48000000);
// // throughput = (width*height) / exec_time_s;

//
// // Double benchmark:
// // //TODO: Record the start time
// // start_time = HAL_GetTick();
// // //TODO: Call the Mandelbrot Function and store the output in the checksum variable defined initially
// // checksum = calculate_mandelbrot_double(width, height, MAX_ITER);
// // //TODO: Record the end time
// // end_time = HAL_GetTick();
// // //TODO: Calculate the execution time
// // execution_time = end_time - start_time;
//
// // execution time in seconds
// // exec_time_s = execution_time / 1000.0;
// // calculate number of cycles
// // cycles = (uint64_t)(exec_time_s*48000000);
// // throughput = (width*height) / exec_time_s;

// //TODO: Visual indicator: Turn on LED1 to signal processing start
// HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1, GPIO_PIN_SET);
//
//
// //TODO: Keep the LEDs ON for 2s
// HAL_Delay(2000);
//
// // TODO: Turn OFF LEDs
// HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0 | GPIO_PIN_1, GPIO_PIN_RESET);
//
// }

// // TASK 7: ===== //
// // for (int i = 0; i < 5; i++) { // image sizes
// // width = image_dimensions[i];
// // height = image_dimensions[i];
// //
// // for (int j = 0; j < 3; j++) {
// // scale = scales[j];
// //
// // // --- FIXED-POINT version ---
// // start_time = HAL_GetTick();
// // checksum = calculate_mandelbrot_fixed_point_arithmetic_custom(width,
height, MAX_ITER, scale);
// // end_time = HAL_GetTick();
// // execution_time = end_time - start_time; // in ms
// // cycles = (uint64_t)(execution_time * 48000000 / 1000);
// // pixels_s = (width * height) / (execution_time / 1000);
// //

```



```

////                                     // --- DOUBLE version ---
////                                     start_time = HAL_GetTick();
////                                     checksum = calculate_mandelbrot_double(256, 256, max_iter);
////                                     end_time = HAL_GetTick();
////                                     execution_time = end_time - start_time; // in ms
//
//                                     HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_0); // blink to show progress
//                                     HAL_Delay(200);
//                                     }
//                                     }
//                                     while(1);

//     // TASK 8: ===== //
//     HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_0);
//     checksum = calculate_mandelbrot_fixed_point_arithmetic(128, 128, MAX_ITER);
//     // checksum = calculate_mandelbrot_fixed_point_arithmetic(256, 256, MAX_ITER);
//     HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_1);
//     HAL_Delay(200);
//     HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_2);

}
/* USER CODE END 3 */
}

/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    /** Initializes the RCC Oscillators according to the specified parameters
     * in the RCC_OscInitTypeDef structure.
     */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
    RCC_OscInitStruct.HSISState = RCC_HSI_ON;
    RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
    RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL12;
    RCC_OscInitStruct.PLL.PREDIV = RCC_PREDIV_DIV1;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
    {
        Error_Handler();
    }

    /** Initializes the CPU, AHB and APB buses clocks
     */
    RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                                   |RCC_CLOCKTYPE_PCLK1;
    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;

    if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_1) != HAL_OK)
    {
        Error_Handler();
    }
}

/**
 * @brief GPIO Initialization Function
 * @param None

```

```

    * @retval None
    */
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    /* USER CODE BEGIN MX_GPIO_Init_1 */

    /* USER CODE END MX_GPIO_Init_1 */

    /* GPIO Ports Clock Enable */
    __HAL_RCC_GPIOF_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();
    __HAL_RCC_GPIOA_CLK_ENABLE();

    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0|GPIO_PIN_1|GPIO_PIN_2|GPIO_PIN_3
        |GPIO_PIN_4|GPIO_PIN_5|GPIO_PIN_6|GPIO_PIN_7, GPIO_PIN_RESET);

    /*Configure GPIO pins : PB0 PB1 PB2 PB3
        PB4 PB5 PB6 PB7 */
    GPIO_InitStruct.Pin = GPIO_PIN_0|GPIO_PIN_1|GPIO_PIN_2|GPIO_PIN_3
        |GPIO_PIN_4|GPIO_PIN_5|GPIO_PIN_6|GPIO_PIN_7;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

    /* USER CODE BEGIN MX_GPIO_Init_2 */

    /* USER CODE END MX_GPIO_Init_2 */
}

/* USER CODE BEGIN 4 */
//TODO: Function signatures you defined previously , implement them here
// FIXED-PT WITH DEFAULT SCALE
uint64_t calculate_mandelbrot_fixed_point_arithmetic(int width, int height, int max_iterations) {
    uint64_t mandelbrot_sum = 0;

    for(int y = 0; y < height; y++){
        for(int x = 0; x < width; x++){
            // convert pixel coordinates to fixed point complex plane coordinates
            int32_t x0 = ((int64_t)x*SCALE*35/(width*10))-(SCALE*25)/10; // making sure all
            numbers are integers not floats
            int32_t y0 = ((int64_t)y * SCALE * 2/ height)-SCALE;

            int32_t xi = 0;
            int32_t yi = 0;
            int i = 0;

            while (i < max_iterations){
                int64_t xi_sq = ((int64_t)xi * xi) / SCALE;
                int64_t yi_sq = ((int64_t)yi * yi) / SCALE;

                if(xi_sq + yi_sq <= 4 * SCALE){
                    int32_t temp = (int32_t)(xi_sq - yi_sq);
                    int64_t xy = ((int64_t)xi * yi) / SCALE;

                    yi = (int32_t)(2*xy + y0);
                    xi = temp + x0;

                    i++;
                }

                else{
                    break;
                }
            }
        }
    }
}

```

```

        }
        mandelbrot_sum += i;
    }
}
return mandelbrot_sum;
}

// FIXED-PT WITH CUSTOM SCALE
uint64_t calculate_mandelbrot_fixed_point_arithmetic_custom(int width, int height, int max_iterations, int scale)
{
    uint64_t mandelbrot_sum = 0;

    for(int y = 0; y < height; y++){
        for(int x = 0; x < width; x++){
            // convert pixel coordinates to fixed point complex plane coordinates
            int32_t x0 = ((int64_t)x*scale*35/(width*10))-(scale*25)/10; // making sure all
numbers are integers not floats
            int32_t y0 = ((int64_t)y * scale * 2/ height)-scale;

            int32_t xi = 0;
            int32_t yi = 0;
            int i = 0;

            while (i < max_iterations){
                int64_t xi_sq = ((int64_t)xi * xi) / scale;
                int64_t yi_sq = ((int64_t)yi * yi) / scale;

                if(xi_sq + yi_sq <= 4 * scale){
                    int32_t temp = (int32_t)(xi_sq - yi_sq);
                    int64_t xy = ((int64_t)xi * yi) / scale;

                    yi = (int32_t)(2*xy + y0);
                    xi = temp + x0;

                    i++;
                }
                else{
                    break;
                }
            }
            mandelbrot_sum += i;
        }
    }
    return mandelbrot_sum;
}

// fixed-pt Mandelbrot calculation for a single tile
uint64_t calculate_mandelbrot_tile_fixed_point(uint16_t tile_x, uint16_t tile_y,
        uint16_t tile_width, uint16_t tile_height,
        uint16_t total_width, uint16_t total_height,
        int max_iterations) {
    uint64_t mandelbrot_sum = 0;

    for(int py = 0; py < tile_height; py++){
        for(int px = 0; px < tile_width; px++){
            // Calculate actual coordinates in the full image
            int actual_x = tile_x + px;
            int actual_y = tile_y + py;

            // Convert pixel coordinates to fixed point complex plane coordinates
            // Using the full image dimensions for coordinate mapping
            int32_t x0 = ((int64_t)actual_x*SCALE*35/(total_width*10))-(SCALE*25)/10;
            int32_t y0 = ((int64_t)actual_y * SCALE * 2/ total_height)-SCALE;

            int32_t xi = 0;

```

```

int32_t yi = 0;
int i = 0;

while (i < max_iterations){
    int64_t xi_sq = ((int64_t)xi * xi) / SCALE;
    int64_t yi_sq = ((int64_t)yi * yi) / SCALE;

    if(xi_sq + yi_sq <= 4 * SCALE){
        int32_t temp = (int32_t)(xi_sq - yi_sq);
        int64_t xy = ((int64_t)xi * yi) / SCALE;

        yi = (int32_t)(2*xy + y0);
        xi = temp + x0;

        i++;
    } else {
        break;
    }
}
mandelbrot_sum += i;
}
}
return mandelbrot_sum;
}

uint64_t calculate_mandelbrot_direct_fixed_point(int width, int height, int max_iterations) {
    return calculate_mandelbrot_fixed_point_arithmetic(width, height, max_iterations);
}

// double precision implementation
uint64_t calculate_mandelbrot_double(int width, int height, int max_iterations){
    uint64_t mandelbrot_sum = 0;

    for (int y = 0; y < height; ++y) {
        double y0 = ((double)y / (double)height) * 2.0 - 1.0;    // map y -> [-1, +1]
        for (int x = 0; x < width; ++x) {
            double x0 = ((double)x / (double)width) * 3.5 - 2.5; // map x -> [-2.5, 1.0]
            double xi = 0.0;
            double yi = 0.0;
            int i = 0;

            while (i < max_iterations && (xi*xi + yi*yi) <= 4.0) {
                double tmp = xi*xi - yi*yi + x0;
                yi = 2.0 * xi * yi + y0;
                xi = tmp;
                ++i;
            }
            mandelbrot_sum += (uint64_t)i;
        }
    }
    return mandelbrot_sum;
}

// for task 4: test if can allocate memory for a given size
bool can_allocate_image(uint16_t width, uint16_t height) {
    uint32_t bytes_needed = (uint32_t)width * height * sizeof(uint32_t);

    const uint32_t SRAM_LIMIT = 8 * 1024; // 8 KB
    // const uint32_t SRAM_LIMIT = 128 * 1024; // 128 KB

    // Leave some margin for stack/heap (say 75%)
    return (bytes_needed <= (SRAM_LIMIT * 3) / 4);
}

```

/* USER CODE END 4 */

```

/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error return state */
    __disable_irq();
    while (1)
    {
    }
    /* USER CODE END Error_Handler_Debug */
}

#ifdef USE_FULL_ASSERT
/**
 * @brief Reports the name of the source file and the source line number
 *        where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number
 * @retval None
 */
void assert_failed(uint8_t *file, uint32_t line)
{
    /* USER CODE BEGIN 6 */
    /* User can add his own implementation to report the file name and line number,
     ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
    /* USER CODE END 6 */
}

#endif /* USE_FULL_ASSERT */

```

Code for **STM32F0**:

```
/* USER CODE BEGIN Header */
/**
 * *****
 * @file      : main.c
 * @brief     : Main program body
 * *****
 * @attention
 *
 * Copyright (c) 2025 STMicroelectronics.
 * All rights reserved.
 *
 * This software is licensed under terms that can be found in the LICENSE file
 * in the root directory of this software component.
 * If no LICENSE file comes with this software, it is provided AS-IS.
 *
 * *****
 */
/* USER CODE END Header */
/* Includes -----*/
#include "main.h"
#include <stdlib.h>
#include <stdbool.h>

/* Private includes -----*/
/* USER CODE BEGIN Includes */

/* USER CODE END Includes */

/* Private typedef -----*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define -----*/
/* USER CODE BEGIN PD */

/* USER CODE END PD */

/* Private macro -----*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables -----*/

/* USER CODE BEGIN PV */
//TODO: Define variables you think you might need
// - Performance timing variables (e.g execution time, throughput, pixels per second, clock cycles)

//task 1 - port from practical 1B
#define MAX_ITER 100 // INITIAL ITERATIONS VALUE
#define SCALE 4096 // INITIAL FIXED-PT SCALE
#define MAX_W 1920
#define MAX_H 1080
#define MAX_TILE 128

uint32_t start_time = 0;
uint32_t end_time = 0;
uint32_t execution_time = 0;
uint64_t checksum = 0;
uint32_t start_cycles = 0;
uint32_t elapsed_cycles = 0;
uint32_t pixels_s = 0;

int32_t height = 0;
```

```

int32_t width = 0;

int32_t max_iter = 0; // for task 2
int32_t scale = 0; // for task 7

const int scales[] = {1000, 10000, 1000000};
const int max_iter_values[] = {100, 250, 500, 750, 1000};
const int image_dimensions[] = {128, 160, 192, 224, 256};

// for task 4
typedef struct {
    uint16_t width;
    uint16_t height;
} image_size_t;

typedef struct {
    uint16_t start_x, start_y;
    uint16_t width, height;
} tile_t;

typedef struct {
    uint16_t width, height;
    uint32_t execution_time_ms;
    uint64_t checksum;
    uint32_t throughput_pixels_per_sec;
    bool used_tiling;
    uint16_t num_tiles;
} performance_result_t;

image_size_t test_sizes[] = {
    {128, 128}, // from prac 1B
    {256, 256}, // from prac 1B
    {320, 240}, // QVGA
    {640, 480}, // VGA
    {800, 600}, // SVGA
    {1024, 768}, // XGA
    {1280, 720}, // HD
    {1920, 1080} // Full HD
};

#define NUM_TEST_SIZES (sizeof(test_sizes)/sizeof(test_sizes[0]))

performance_result_t results[NUM_TEST_SIZES];
uint8_t result_count = 0;

/* USER CODE END PV */

/* Private function prototypes -----*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
/* USER CODE BEGIN PFP */
//TODO: Define any function prototypes you might need such as the calculate Mandelbrot function among others
uint64_t calculate_mandelbrot_fixed_point_arithmetic(int width, int height, int max_iterations);
uint64_t calculate_mandelbrot_fixed_point_arithmetic_custom(int width, int height, int max_iterations, int32_t scale);
uint64_t calculate_mandelbrot_double(int width, int height, int max_iterations);
uint64_t calculate_mandelbrot_float(int width, int height, int max_iterations);
uint64_t calculate_mandelbrot_tile_fixed_point(uint16_t tile_x, uint16_t tile_y,
        uint16_t tile_width, uint16_t tile_height,
        uint16_t total_width, uint16_t total_height,
        int max_iterations);

uint64_t calculate_mandelbrot_direct_fixed_point(int width, int height, int max_iterations);
bool can_allocate_image(uint16_t width, uint16_t height);

/* USER CODE END PFP */

```

```

/* Private user code -----*/
/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

/**
 * @brief The application entry point.
 * @retval int
 */
int main(void)
{

/* USER CODE BEGIN 1 */

/* USER CODE END 1 */

/* MCU Configuration-----*/

/* Reset of all peripherals, Initializes the Flash interface and the Systick. */
HAL_Init();

/* USER CODE BEGIN Init */

/* USER CODE END Init */

/* Configure the system clock */
SystemClock_Config();

/* USER CODE BEGIN SysInit */

/* USER CODE END SysInit */

/* Initialize all configured peripherals */
MX_GPIO_Init();
/* USER CODE BEGIN 2 */

// DWT cycle counter setup
CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk; // Enable trace and debug
DWT->CYCCNT = 0; // Reset counter
DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk; // Enable cycle counter

/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
// TASK 2: ===== //
for (int i = 0; i < 5; i++) { // image sizes
width = image_dimensions[i];
height = image_dimensions[i];
//
for (int j = 0; j < 5; j++) { // MAX_ITER values
max_iter = max_iter_values[j];
//
// --- FIXED-POINT version ---
start_cycles = DWT->CYCCNT;
checksum = calculate_mandelbrot_fixed_point_arithmetic(width, height, max_iter);
elapsed_cycles = DWT->CYCCNT - start_cycles;
execution_time = (double)elapsed_cycles / ((double)SystemCoreClock/1000); // in ms
//
// --- DOUBLE version ---
start_cycles = DWT->CYCCNT;
checksum = calculate_mandelbrot_double(width, height, max_iter);
elapsed_cycles = DWT->CYCCNT - start_cycles;
}
}
}

```



```

//          execution_time = (double)elapsed_cycles / ((double)SystemCoreClock/1000); // in ms
//
//          HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_0); // blink to show progress
//          HAL_Delay(200);
//      }
//  }
//  while(1); // stop after one sweep

// TASK 4: ===== //
for (int i = 0; i < sizeof(test_sizes)/sizeof(test_sizes[0]); i++) {
    width = test_sizes[i].width;
    height = test_sizes[i].height;

    bool use_tiling = !can_allocate_image(width, height);

    start_cycles = DWT->CYCCNT;
    if (!use_tiling) {
        // direct computation if memory is OK
        checksum = calculate_mandelbrot_direct_fixed_point(width, height, MAX_ITER);
    } else {
        // break into tiles
        checksum = 0;
        uint16_t num_tiles_x = (width + MAX_TILE - 1) / MAX_TILE;
        uint16_t num_tiles_y = (height + MAX_TILE - 1) / MAX_TILE;

        for (int ty = 0; ty < num_tiles_y; ty++) {
            for (int tx = 0; tx < num_tiles_x; tx++) {
                uint16_t tile_w = ((tx+1)*MAX_TILE > width) ? (width - tx*MAX_TILE) : MAX_TILE;
                uint16_t tile_h = ((ty+1)*MAX_TILE > height) ? (height - ty*MAX_TILE) : MAX_TILE;

                checksum += calculate_mandelbrot_tile_fixed_point(
                    tx*MAX_TILE, ty*MAX_TILE,
                    tile_w, tile_h,
                    width, height, MAX_ITER);
            }
        }
    }
    elapsed_cycles = DWT->CYCCNT - start_cycles;
    execution_time = (uint32_t)((double)elapsed_cycles / ((double)SystemCoreClock/1000)); // ms
    pixels_s = (uint32_t)((double)(width*height) / (execution_time / 1000.0));

    // optional: blink LED as progress indicator
    HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_0);
    HAL_Delay(100);

    // === RECORD RESULT ===
    results[result_count].width = width;
    results[result_count].height = height;
    results[result_count].execution_time_ms = execution_time;
    results[result_count].checksum = checksum;
    results[result_count].throughput_pixels_per_sec = pixels_s;
    results[result_count].used_tiling = use_tiling;
    results[result_count].num_tiles = use_tiling ?
        ((width + MAX_TILE - 1)/MAX_TILE) * ((height + MAX_TILE - 1)/MAX_TILE) : 1;

    result_count++;
}
while(1); // stop after one sweep

// TASK 5: ===== //
// /* USER CODE END WHILE */
// for (int i = 0; i < 5; i++) {
//     width = image_dimensions[i];

```

```

//          height = image_dimensions[i];
//
//  /* USER CODE BEGIN 3 */
//      //TODO: Visual indicator: Turn on LED0 to signal processing start
//      HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET);
//
//      /* commenting out fixed-point implementation for task 5
//      //TODO: Benchmark and Profile Performance
//      // Fixed-point benchmark:
//      //TODO: Record the start number of cycles
//      start_cycles = DWT->CYCCNT;
//      //TODO: Call the Mandelbrot Function and store the output in the checksum variable defined initially
//      checksum = calculate_mandelbrot_fixed_point_arithmetic(width, height, MAX_ITER);
//      //TODO: Calculate elapsed cycles
//      elapsed_cycles = DWT->CYCCNT - start_cycles;
//      // convert cycles to seconds
//      execution_time = elapsed_cycles / SystemCoreClock; //SystemCoreClock => HAL variable that stores CPU
frequency (Hz)
//
//  */
//
//  // Double benchmark:
//  //TODO: Record the start number of cycles
//  start_cycles = DWT->CYCCNT;
//  //TODO: Call the Mandelbrot Function and store the output in the checksum variable defined initially
//  checksum = calculate_mandelbrot_double(width, height, MAX_ITER);
//  //TODO: Calculate elapsed cycles
//  elapsed_cycles = DWT->CYCCNT - start_cycles;
//  // convert cycles to seconds
//  execution_time = elapsed_cycles / SystemCoreClock;
//
//  /*
//  // floating point benchmark:
//  // record start number of cycles
//  start_cycles = DWT->CYCCNT;
//  // call Mandelbrot Function and store the output in the checksum variable
//  checksum = calculate_mandelbrot_float(width, height, MAX_ITER);
//  // calculate elapsed cycles
//  elapsed_cycles = DWT->CYCCNT - start_cycles;
//  //convert cycles to seconds
//  execution_time = elapsed_cycles / SystemCoreClock;
//  */
//
//      //TODO: Visual indicator: Turn on LED1 to signal processing start
//      HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1, GPIO_PIN_SET);
//
//      //TODO: Keep the LEDs ON for 2s
//      HAL_Delay(2000);
//
//      //TODO: Turn OFF LEDs
//      HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0 | GPIO_PIN_1, GPIO_PIN_RESET);
//      }

//  // TASK 7: ===== //
//      for (int i = 0; i < 5; i++) {          // image sizes
//          width = image_dimensions[i];
//          height = image_dimensions[i];
//
//          for (int j = 0; j < 3; j++) {
//              scale = scales[j];
//
//              // --- FIXED-POINT version ---
//              start_cycles = DWT->CYCCNT;
//              checksum = calculate_mandelbrot_fixed_point_arithmetic_custom(width, height,
MAX_ITER, scale);

```

```

//                                     elapsed_cycles = DWT->CYCCNT - start_cycles;
//                                     execution_time = (double)elapsed_cycles / ((double)SystemCoreClock/1000); // in
ms
//                                     pixels_s = (width * height) / (execution_time / 1000);
//
////                                     // --- DOUBLE version ---
////                                     start_cycles = DWT->CYCCNT;
////                                     checksum = calculate_mandelbrot_double(width, height, MAX_ITER);
////                                     elapsed_cycles = DWT->CYCCNT - start_cycles;
////                                     execution_time = (double)elapsed_cycles / ((double)SystemCoreClock/1000); // in
ms
////                                     pixels_s = (width * height) / (execution_time * 1000);
//
//                                     HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_0); // blink to show progress
//                                     HAL_Delay(200);
//                                     }
//                                     }
//                                     while(1); // stop after one sweep

//      // TASK 8: ===== //
//      HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_0);
//      checksum = calculate_mandelbrot_fixed_point_arithmetic(128, 128, MAX_ITER);
//      // checksum = calculate_mandelbrot_fixed_point_arithmetic(256, 256, MAX_ITER);
//      HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_1);
//      HAL_Delay(200);
//      HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_2);

}
/* USER CODE END 3 */
}

/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    /** Configure the main internal regulator output voltage
    */
    __HAL_RCC_PWR_CLK_ENABLE();
    __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE3);

    /** Initializes the RCC Oscillators according to the specified parameters
    * in the RCC_OscInitTypeDef structure.
    */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
    RCC_OscInitStruct.HSEState = RCC_HSE_ON;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
    RCC_OscInitStruct.PLL.PLLM = 15;
    RCC_OscInitStruct.PLL.PLLN = 144;
    RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
    RCC_OscInitStruct.PLL.PLLQ = 2;
    RCC_OscInitStruct.PLL.PLLR = 2;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
    {
        Error_Handler();
    }

    /** Initializes the CPU, AHB and APB buses clocks
    */

```

```

RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLOCK
    |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV4;
RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV2;

if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_3) != HAL_OK)
{
    Error_Handler();
}
}

/**
 * @brief GPIO Initialization Function
 * @param None
 * @retval None
 */
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    /* USER CODE BEGIN MX_GPIO_Init_1 */

    /* USER CODE END MX_GPIO_Init_1 */

    /* GPIO Ports Clock Enable */
    __HAL_RCC_GPIOC_CLK_ENABLE();
    __HAL_RCC_GPIOH_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();

    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0|GPIO_PIN_1|GPIO_PIN_2|GPIO_PIN_3
        |GPIO_PIN_4|GPIO_PIN_5|GPIO_PIN_6|GPIO_PIN_7, GPIO_PIN_RESET);

    /*Configure GPIO pins : PB0 PB1 PB2 PB3
        PB4 PB5 PB6 PB7 */
    GPIO_InitStruct.Pin = GPIO_PIN_0|GPIO_PIN_1|GPIO_PIN_2|GPIO_PIN_3
        |GPIO_PIN_4|GPIO_PIN_5|GPIO_PIN_6|GPIO_PIN_7;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

    /* USER CODE BEGIN MX_GPIO_Init_2 */

    /* USER CODE END MX_GPIO_Init_2 */
}

/* USER CODE BEGIN 4 */
//TODO: Function signatures you defined previously , implement them here
//fixed point implementation
// MAX_ITER=4096
uint64_t calculate_mandelbrot_fixed_point_arithmetic(int width, int height, int max_iterations) {
    uint64_t mandelbrot_sum = 0;

    for(int y = 0; y < height; y++){
        for(int x = 0; x < width; x++){
            // convert pixel coordinates to fixed point complex plane coordinates
            int32_t x0 = ((int64_t)x*SCALE*35/(width*10))-(SCALE*25)/10; // making sure all
numbers are integers not floats

            int32_t y0 = ((int64_t)y * SCALE * 2/ height)-SCALE;

            int32_t xi = 0;
            int32_t yi = 0;
            int i = 0;

```

```

        while (i < max_iterations){
            int64_t xi_sq = ((int64_t)xi * xi) / SCALE;
            int64_t yi_sq = ((int64_t)yi * yi) / SCALE;

            if(xi_sq + yi_sq <= 4 * SCALE){
                int32_t temp = (int32_t)(xi_sq - yi_sq);
                int64_t xy = ((int64_t)xi * yi) / SCALE;

                yi = (int32_t)(2*xy + y0);
                xi = temp + x0;

                i++;
            }
            else{
                break;
            }
        }
        mandelbrot_sum += i;
    }
}
return mandelbrot_sum;
}

uint64_t calculate_mandelbrot_fixed_point_arithmetic_custom(int width, int height, int max_iterations, int32_t
scale) {
    uint64_t mandelbrot_sum = 0;

    for(int y = 0; y < height; y++){
        for(int x = 0; x < width; x++){
            // convert pixel coordinates to fixed point complex plane coordinates
            int32_t x0 = ((int64_t)x*scale*35/(width*10))-(scale*25)/10; // making sure all
numbers are integers not floats
            int32_t y0 = ((int64_t)y * scale * 2/ height)-scale;

            int32_t xi = 0;
            int32_t yi = 0;
            int i = 0;

            while (i < max_iterations){
                int64_t xi_sq = ((int64_t)xi * xi) / scale;
                int64_t yi_sq = ((int64_t)yi * yi) / scale;

                if(xi_sq + yi_sq <= 4 * scale){
                    int32_t temp = (int32_t)(xi_sq - yi_sq);
                    int64_t xy = ((int64_t)xi * yi) / scale;

                    yi = (int32_t)(2*xy + y0);
                    xi = temp + x0;

                    i++;
                }
                else{
                    break;
                }
            }
            mandelbrot_sum += i;
        }
    }
    return mandelbrot_sum;
}

// fixed-pt Mandelbrot calculation for a single tile
uint64_t calculate_mandelbrot_tile_fixed_point(uint16_t tile_x, uint16_t tile_y,

```

uint16_t

uint16_t

int

```

tile_width, uint16_t tile_height,

total_width, uint16_t total_height,

max_iterations) {
    uint64_t mandelbrot_sum = 0;

    for(int py = 0; py < tile_height; py++){
        for(int px = 0; px < tile_width; px++){
            // Calculate actual coordinates in the full image
            int actual_x = tile_x + px;
            int actual_y = tile_y + py;

            // Convert pixel coordinates to fixed point complex plane coordinates
            // Using the full image dimensions for coordinate mapping
            int32_t x0 = ((int64_t)actual_x*SCALE*35/(total_width*10))-(SCALE*25)/10;
            int32_t y0 = ((int64_t)actual_y * SCALE * 2/ total_height)-SCALE;

            int32_t xi = 0;
            int32_t yi = 0;
            int i = 0;

            while (i < max_iterations){
                int64_t xi_sq = ((int64_t)xi * xi) / SCALE;
                int64_t yi_sq = ((int64_t)yi * yi) / SCALE;

                if(xi_sq + yi_sq <= 4 * SCALE){
                    int32_t temp = (int32_t)(xi_sq - yi_sq);
                    int64_t xy = ((int64_t)xi * yi) / SCALE;

                    yi = (int32_t)(2*xy + y0);
                    xi = temp + x0;

                    i++;
                } else {
                    break;
                }
            }
            mandelbrot_sum += i;
        }
    }
    return mandelbrot_sum;
}

// Direct processing (when memory allows)
uint64_t calculate_mandelbrot_direct_fixed_point(int width, int height, int max_iterations) {
    return calculate_mandelbrot_tile_fixed_point(0, 0, width, height, width, height, max_iterations);
}

//double precision implementation
uint64_t calculate_mandelbrot_double(int width, int height, int max_iterations){
    uint64_t mandelbrot_sum = 0;
    for (int y = 0; y < height; ++y) {
        double y0 = ((double)y / (double)height) * 2.0 - 1.0;    // map y -> [-1, +1]
        for (int x = 0; x < width; ++x) {
            double x0 = ((double)x / (double)width) * 3.5 - 2.5; // map x -> [-2.5, 1.0]
            double xi = 0.0;
            double yi = 0.0;
            int i = 0;

            while (i < max_iterations && (xi*xi + yi*yi) <= 4.0) {
                double tmp = xi*xi - yi*yi + x0;
                yi = 2.0 * xi * yi + y0;
                xi = tmp;
            }
        }
    }
    return mandelbrot_sum;
}

```

```

        ++i;
    }
    mandelbrot_sum += (uint64_t)i;
}
}
return mandelbrot_sum;
}

//float implementation
uint64_t calculate_mandelbrot_float(int width, int height, int max_iterations){
    uint64_t mandelbrot_sum = 0;
    for(int y = 0; y < height; y++){
        float y0 = ((float)y / (float)height) * 2.0f - 1.0f;
        for(int x = 0; x < width; ++x){
            float x0 = ((float)x / (float)width) * 3.5f - 2.5f;
            float xi = 0.0f;
            float yi = 0.0f;
            int i = 0;

            while (i < max_iterations && (xi*xi + yi*yi) <= 4.0f){
                float tmp = xi*xi - yi*yi + x0;
                yi = 2.0f * xi * yi + y0;
                xi = tmp;
                ++i;
            }
            mandelbrot_sum += (uint64_t)i;
        }
    }
    return mandelbrot_sum;
}

```

```

// for task 4: test if can allocate memory for a given size
bool can_allocate_image(uint16_t width, uint16_t height) {
    uint32_t bytes_needed = (uint32_t)width * height * sizeof(uint32_t);

    // const uint32_t SRAM_LIMIT = 8 * 1024; // 8 KB
    const uint32_t SRAM_LIMIT = 128 * 1024; // 128 KB

    // Leave some margin for stack/heap (say 75%)
    return (bytes_needed <= (SRAM_LIMIT * 3) / 4);
}

```

/* USER CODE END 4 */

```

/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error return state */
    __disable_irq();
    while (1)
    {
    }
    /* USER CODE END Error_Handler_Debug */
}

#ifdef USE_FULL_ASSERT
/**
 * @brief Reports the name of the source file and the source line number
 * where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number
 * @retval None

```

```
*/
void assert_failed(uint8_t *file, uint32_t line)
{
    /* USER CODE BEGIN 6 */
    /* User can add his own implementation to report the file name and line number,
       ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
    /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */
```